create three objects, each with different initial state. The p method prints out an internal representation of an object. Using it, we can see that in each case our parameters got transferred into the object's state, ending up as instance variables:

tut_classes/stock_stats/book_in_stock_1.rb

```
class BookInStock
  def initialize(isbn, price)
    @isbn = isbn
    @price = Float(price)
  end
end

b1 = BookInStock.new("isbn1", 3)
p b1

b2 = BookInStock.new("isbn2", 3.14)
p b2

b3 = BookInStock.new("isbn3", "5.67")
p b3
```

produces:

```
#<BookInStock:0x0000000100a188c0 @isbn="isbn1", @price=3.0>
#<BookInStock:0x0000000100a18410 @isbn="isbn2", @price=3.14>
#<BookInStock:0x0000000100a182a8 @isbn="isbn3", @price=5.67>
```

Why did we use the p method to write out our objects, rather than puts? Well, let's repeat the code using puts:

tut_classes/stock_stats/book_in_stock_1a.rb

```
class BookInStock
  def initialize(isbn, price)
    @isbn = isbn
    @price = Float(price)
  end
end

b1 = BookInStock.new("isbn1", 3)
puts b1

b2 = BookInStock.new("isbn2", 3.14)
puts b2

b3 = BookInStock.new("isbn3", "5.67")
puts b3
```

produces:

```
#<BookInStock:0x0000000102c587c8>
#<BookInStock:0x0000000102c583e0>
#<BookInStock:0x0000000102c58318>
```

Remember, puts writes strings to your program's standard output. When you pass it an object based on a class you wrote, it doesn't really know what to do with the object yet, so it uses a very simple expedient: it writes the name of the object's class, followed by a colon and the object's unique object identifier, which is a hexadecimal number. It puts the whole lot inside #<...>.

Our experience tells us that during development we'll be printing out the contents of a BookInStock object many times, and the default formatting leaves something to be desired.